

Computer Vision Image Filtering

Lectures: Kira Maag

Exercises: Arnold Brosch & Ben Hamscher

Visual Analytics

Computer Science Department

Heinrich-Heine-University Düsseldorf

`kira.maag@hhu.de` & `arnold.brosch@hhu.de` & `ben.hamscher@hhu.de`

Summer Semester 2026

Table of Content

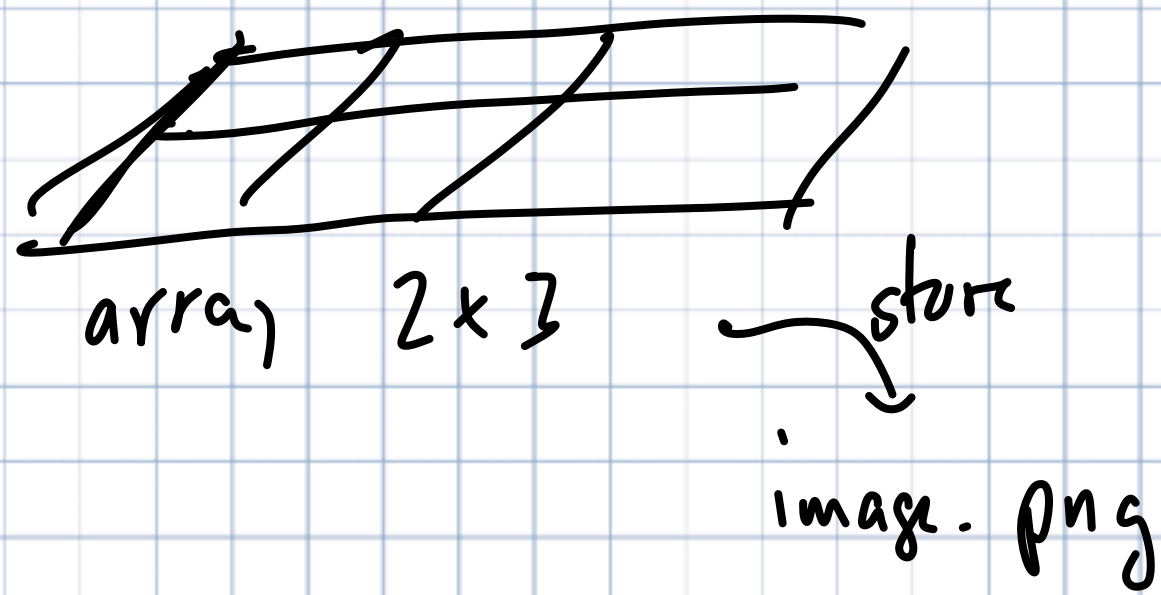
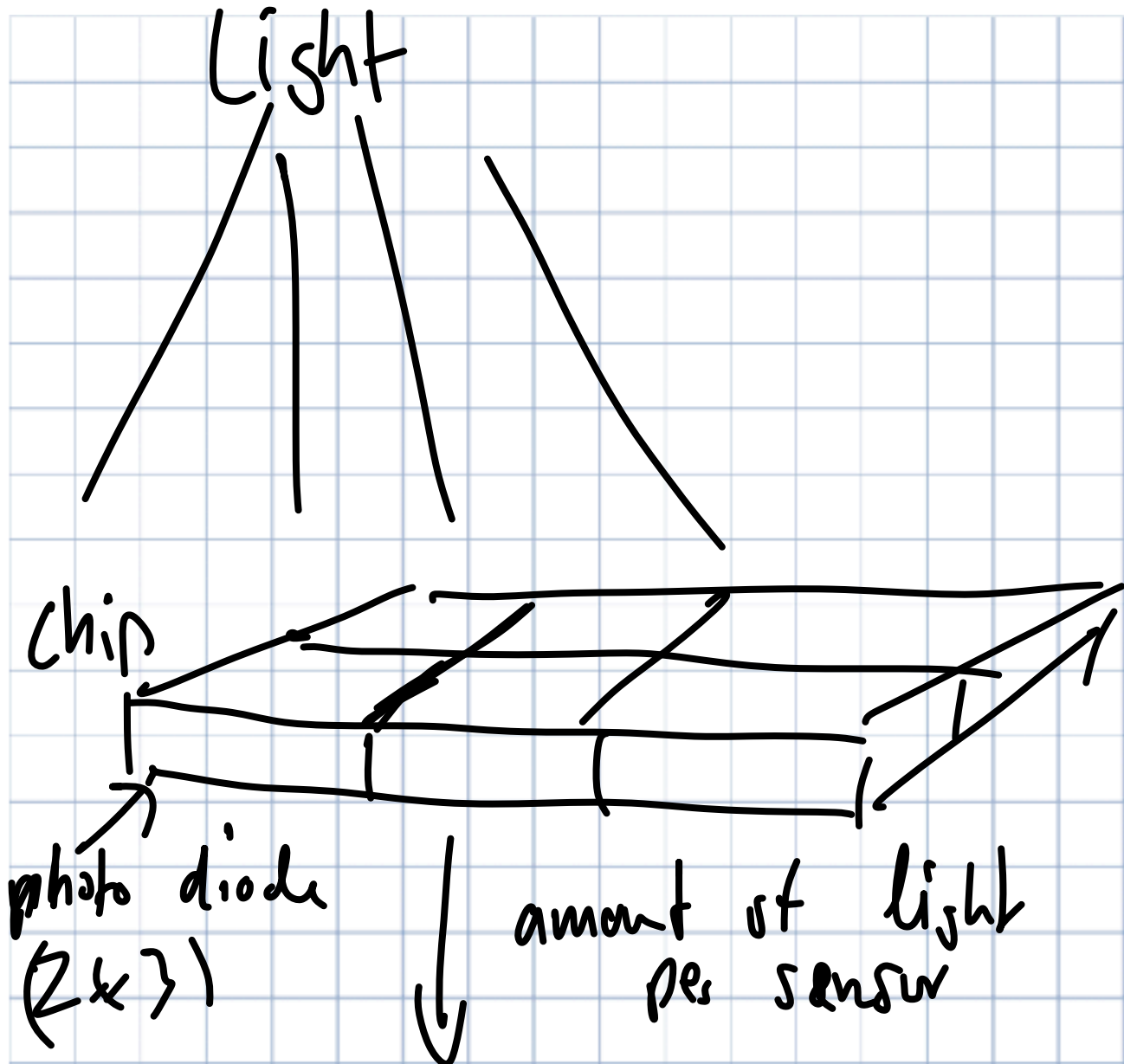
- ▶ Organization:
 - ▶ Please participate in the survey about the final exam
 - ▶ Information about the exam next week
- ▶ Recap - image formation:
 - ▶ Continue Jupyter Notebook
 - ▶ Sample calculations of coordinate transformations
- ▶ Today - image filtering:
 - ▶ Pixel intensities
 - ▶ Linear filters

Last lecture

What is an image?

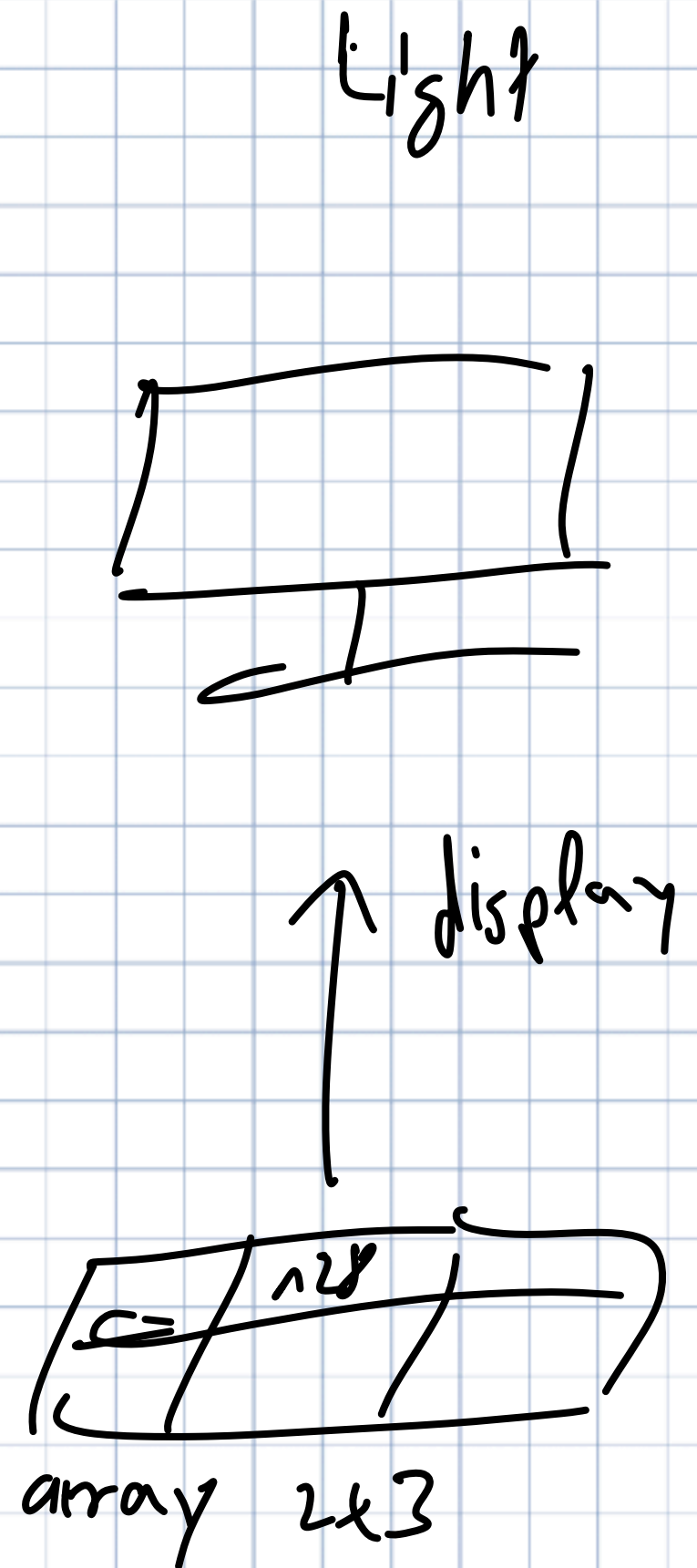
We consider an image as a mapping from a D -dimensional domain of coordinates to a C -dimensional domain of measured intensities:

$$f : \mathbb{R}^D \rightarrow \mathbb{R}^C$$



load

An arrow points from the "load" text towards the right, indicating the direction of data retrieval from storage.



Pixels

A „pixel“ (picture element) is a point in an image, characterized by

- its position $\mathbf{x} = [x_1 \ \dots \ x_D]^\top \in \mathbb{R}^D$
- its value $\mathbf{v} = [v_1 \ \dots \ v_C]^\top \in \mathbb{R}^C$

It can thus be considered a tuple

$$p = (\mathbf{x}, \mathbf{v})$$

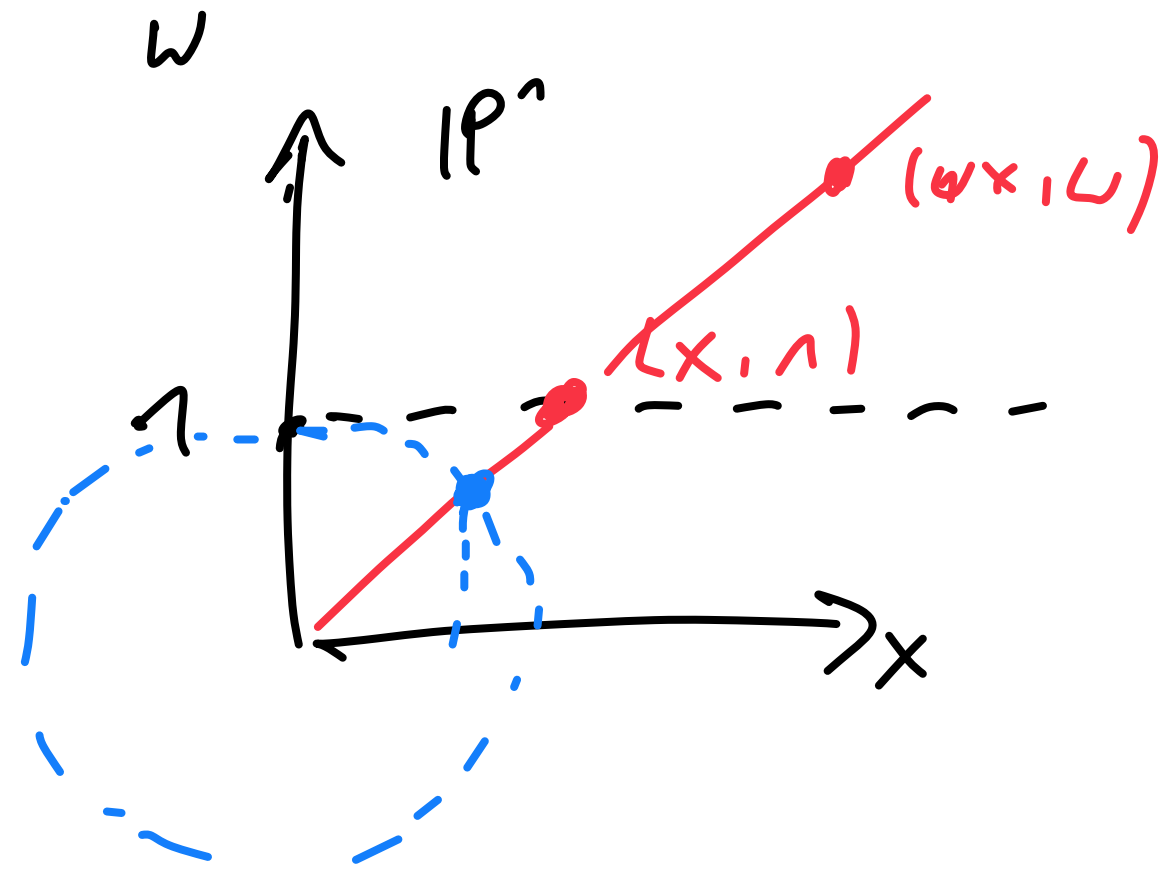
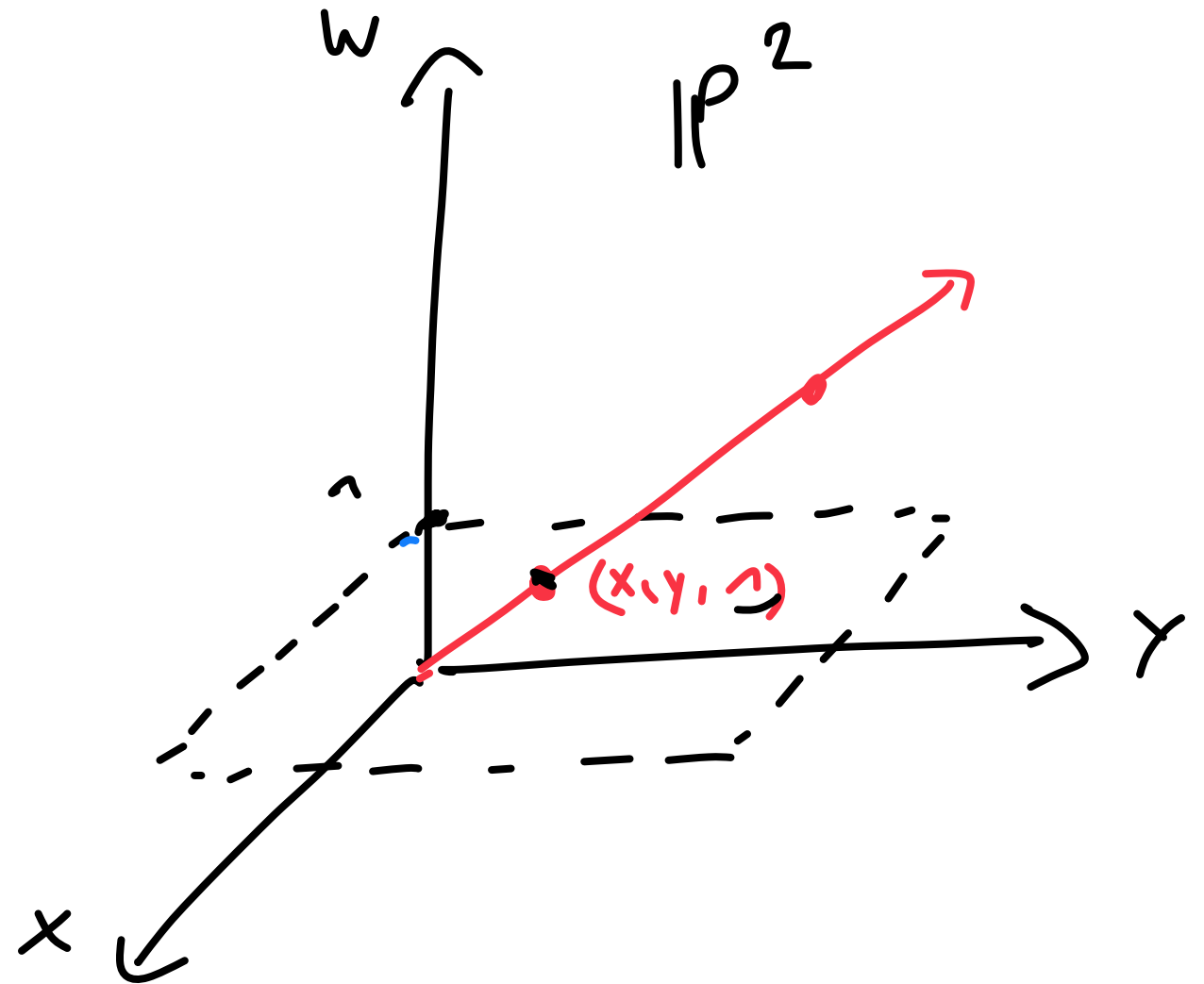
2D points

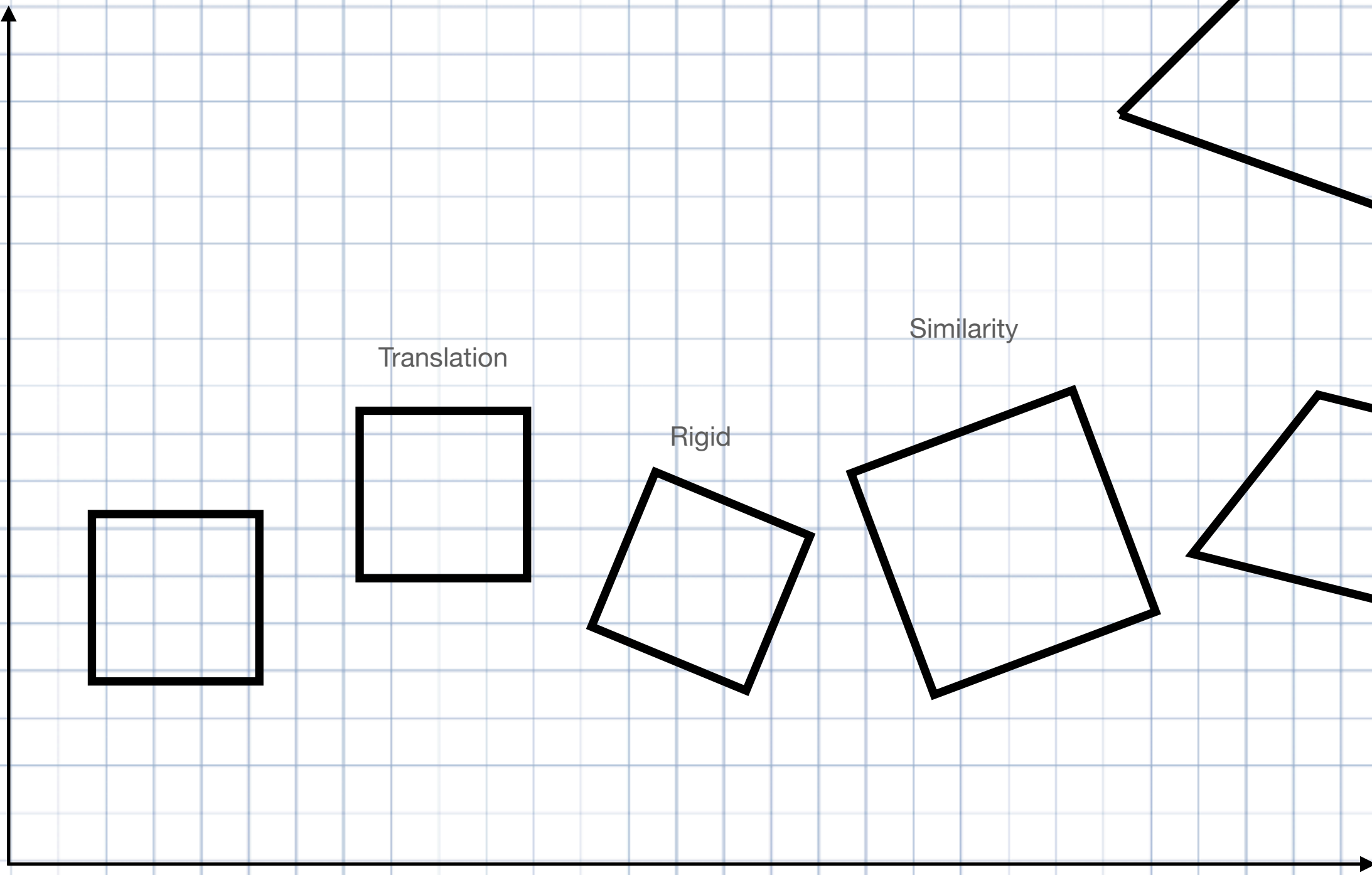
The 2D point $\mathbf{x} = (x, y) = [x \ y]^T$ can be represented in *homogenous coordinates*

$$\tilde{\mathbf{x}} = (\tilde{x}, \tilde{y}, \tilde{w}) = \tilde{w}(x, y, 1) = \tilde{w}\bar{\mathbf{x}} \in \mathbb{P}^2$$

where

- scaled points are identical
- dividing by the 3rd coordinate gives the original Euclidean point
- dividing by the length gives us the point on the sphere
- What does $\tilde{w} = 0$ mean?





Translation

Rigid

Similarity

Affine

Perspective

2D translation

The 2D coordinate translation

$$\mathbf{x}' = \mathbf{x} + \mathbf{t} = \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} t_x \\ t_y \end{bmatrix}$$

can be written in homogenous coordinates as

$$\bar{\mathbf{x}}' = \begin{bmatrix} \mathbf{I}_{2 \times 2} & \mathbf{t} \\ \mathbf{0}_{1 \times 2} & 1 \end{bmatrix} \cdot \bar{\mathbf{x}} = \mathbf{A}_t \bar{\mathbf{x}} \quad , \quad \mathbf{A}_t \in \mathbb{R}^{3 \times 3}$$

Note that this holds for any homogenous coordinates:

$$\tilde{\mathbf{x}}' = \begin{bmatrix} \mathbf{I}_{2 \times 2} & \mathbf{t} \\ \mathbf{0}_{1 \times 2} & 1 \end{bmatrix} \cdot \tilde{\mathbf{x}}$$

2D rotation

We can use the 2D rotation matrix

$$\mathbf{R}_\theta = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$$

to rotate a 2D point around the origin by θ via matrix multiplication.

Note: $\mathbf{R}$ is orthonormal, thus

- $\|\mathbf{R}\|^2 = 1$
- $\mathbf{R}\mathbf{R}^\top = \mathbf{I}$

Rigid body transformation

We can perform the joint rotation and translation operation $\mathbf{x}' = \mathbf{R}\mathbf{x} + \mathbf{t}$ using the rigid body transformation matrix

$$\tilde{\mathbf{x}}' = \begin{bmatrix} \mathbf{R}_\theta & \mathbf{t} \\ \mathbf{0}_{1 \times 2} & 1 \end{bmatrix} \tilde{\mathbf{x}}$$

This transformation preserves distances.

Similarity transformation

We can allow an additional scaling s to obtain

$$\tilde{\mathbf{x}}' = \begin{bmatrix} s\mathbf{R}_\theta & \mathbf{t} \\ \mathbf{0}_{1 \times 2} & 1 \end{bmatrix} \tilde{\mathbf{x}}$$

This transformation allows to „resize“ objects, but still preserves angles.

Affine transformation

The linear transformation also allows for shearing using unconstrained upper rows

$$\tilde{\mathbf{x}}' = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ 0 & 0 & 1 \end{bmatrix} \tilde{\mathbf{x}}$$

This transformation does not preserve angles any more, but still preserves parallel lines.

Perspective transformation

Using the full 3×3 matrix, we obtain the perspective transformation

$$\tilde{\mathbf{x}}' = \tilde{\mathbf{H}} \cdot \tilde{\mathbf{x}} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \tilde{\mathbf{x}}$$

$\tilde{\mathbf{H}}$ only preserves straight lines.

It requires homogeneous coordinates.

The Euclidean result requires normalization, thus

$$x' = \frac{h_{11}x + h_{12}y + h_{13}w}{h_{31}x + h_{32}y + h_{33}w} \quad \text{and} \quad y' = \frac{h_{21}x + h_{22}y + h_{23}w}{h_{31}x + h_{32}y + h_{33}w}$$

Transformation	Matrix	# DoF	Preserves	Icon
translation	$\begin{bmatrix} \mathbf{I} & \mathbf{t} \end{bmatrix}_{2 \times 3}$	2	orientation	
rigid (Euclidean)	$\begin{bmatrix} \mathbf{R} & \mathbf{t} \end{bmatrix}_{2 \times 3}$	3	lengths	
similarity	$\begin{bmatrix} s\mathbf{R} & \mathbf{t} \end{bmatrix}_{2 \times 3}$	4	angles	
affine	$\begin{bmatrix} \mathbf{A} \end{bmatrix}_{2 \times 3}$	6	parallelism	
projective	$\begin{bmatrix} \tilde{\mathbf{H}} \end{bmatrix}_{3 \times 3}$	8	straight lines	

Table 2.1 *Hierarchy of 2D coordinate transformations, listing the transformation name, its matrix form, the number of degrees of freedom, what geometric properties it preserves, and a mnemonic icon. Each transformation also preserves the properties listed in the rows below it, i.e., similarity preserves not only angles but also parallelism and straight lines. The 2×3 matrices are extended with a third $[\mathbf{0}^T \ 1]$ row to form a full 3×3 matrix for homogeneous coordinate transformations.*

Today

Today's goals

- Recap 2D coordinate transformations (in a way, „modify pixel locations“)
- Transforming pixel intensities („modify pixel values“)
- Convolution filters
-

Linear intensity transformations - image filtering

Point operations

We may apply functions to modify pixel intensities independently of each other, such as the linear point process

$$g(\mathbf{x}) = aI(\mathbf{x}) + b$$

- a increases contrast (also called „gain“)
- b increases brightness (also called „bias“)
- We can add complexity by making a, b dependent on the image location:

$$g(\mathbf{x}) = a(\mathbf{x})I(\mathbf{x}) + b(\mathbf{x})$$

Then a and b in fact become gray level images themselves!



Let's code a bit!



Linear filters

Given a kernel function $h : \mathbb{R}^D \rightarrow \mathbb{R}^C$, we can filter the image f into a new image g using the convolution operation

$$g(y, x) = \sum_{i,j} f(y - i, x - j) \cdot h(i, j)$$

In short form: $g = f * k$

Note that this refers to discrete images and kernels - in the continuous domain, the convolution uses integrals.

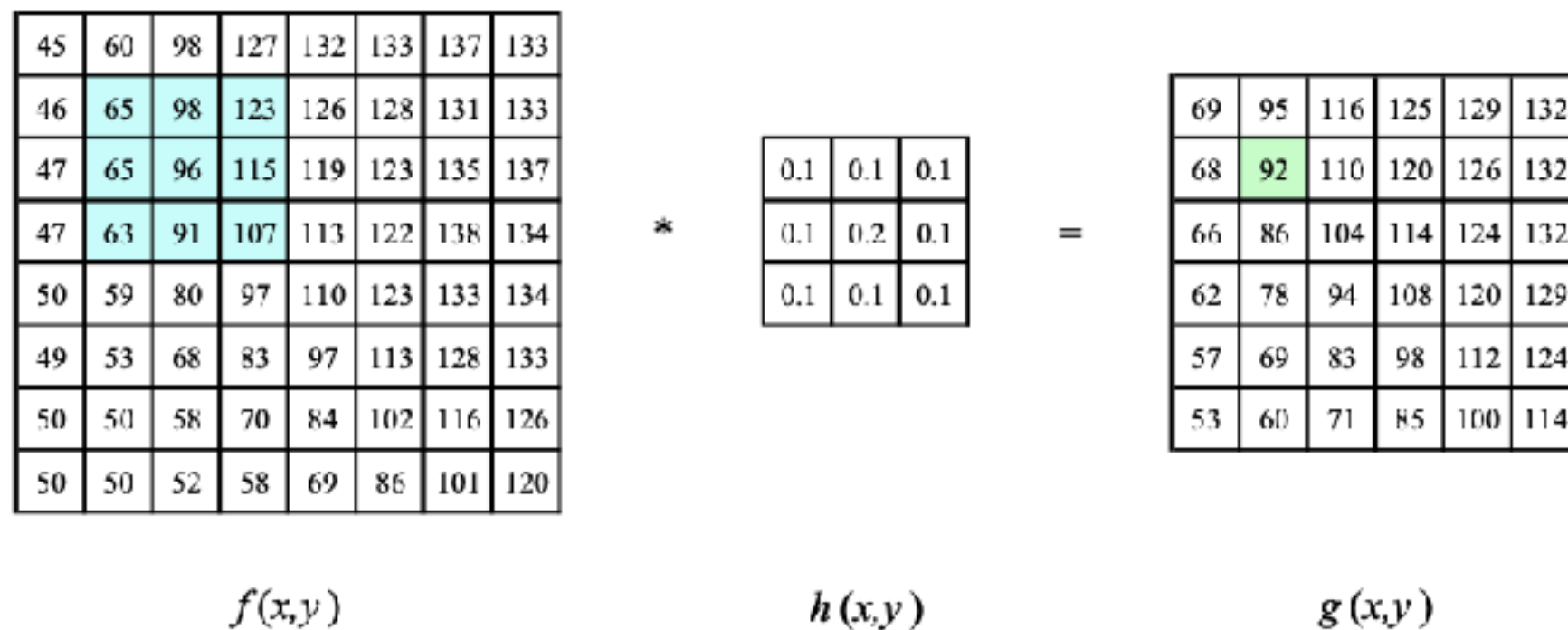


Figure 3.10 *Neighborhood filtering (convolution): The image on the left is convolved with the filter in the middle to yield the image on the right. The light blue pixels indicate the source neighborhood for the light green destination pixel.*

Image filtering

$$g[\cdot, \cdot] \frac{1}{9}$$

1	1	1
1	1	1
1	1	1

$$f[\cdot, \cdot]$$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$$h[\cdot, \cdot]$$

$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

Image filtering

$$g[\cdot, \cdot] \frac{1}{9}$$

1	1	1
1	1	1
1	1	1

 $f[\cdot, \cdot]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

 $h[\cdot, \cdot]$

	0	10	20						

$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

Image filtering

$$g[\cdot, \cdot] \frac{1}{9}$$

1	1	1
1	1	1
1	1	1

$$f[\cdot, \cdot]$$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$$h[\cdot, \cdot]$$

	0	10	20	30	30				

$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

Image filtering

$$g[\cdot, \cdot] \frac{1}{9}$$

1	1	1
1	1	1
1	1	1

$$f[\cdot, \cdot]$$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$$h[\cdot, \cdot]$$

	0	10	20	30	30				
						?			
				50					

$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

Box Filter

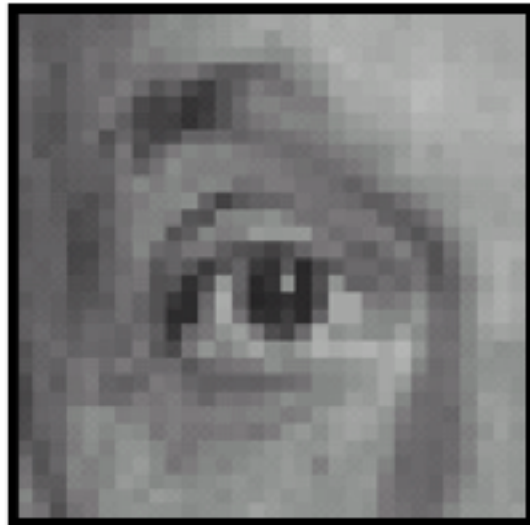
What does it do?

- Replaces each pixel with an average of its neighborhood
- Achieve smoothing effect (remove sharp features)

$$\frac{1}{9} g[\cdot, \cdot]$$

1	1	1
1	1	1
1	1	1

Practice with linear filters

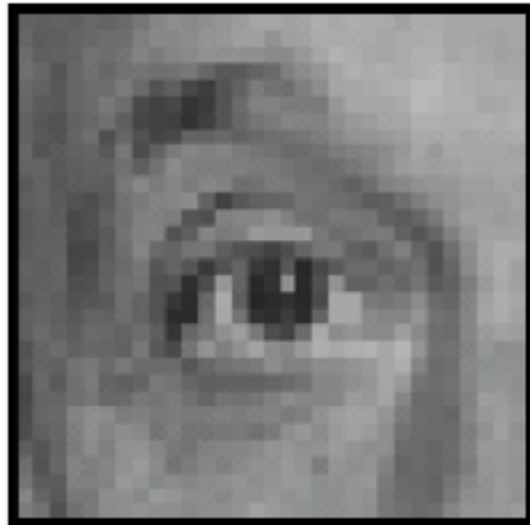


Original

0	0	0
0	1	0
0	0	0

?

Practice with linear filters

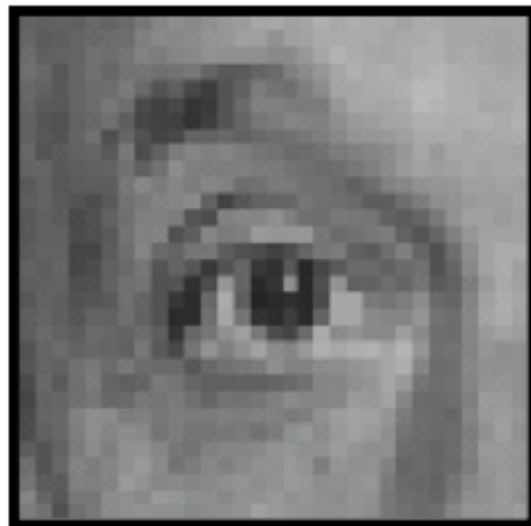


Original

0	0	0
0	0	1
0	0	0

?

Practice with linear filters



Original

0	0	0
0	2	0
0	0	0

−

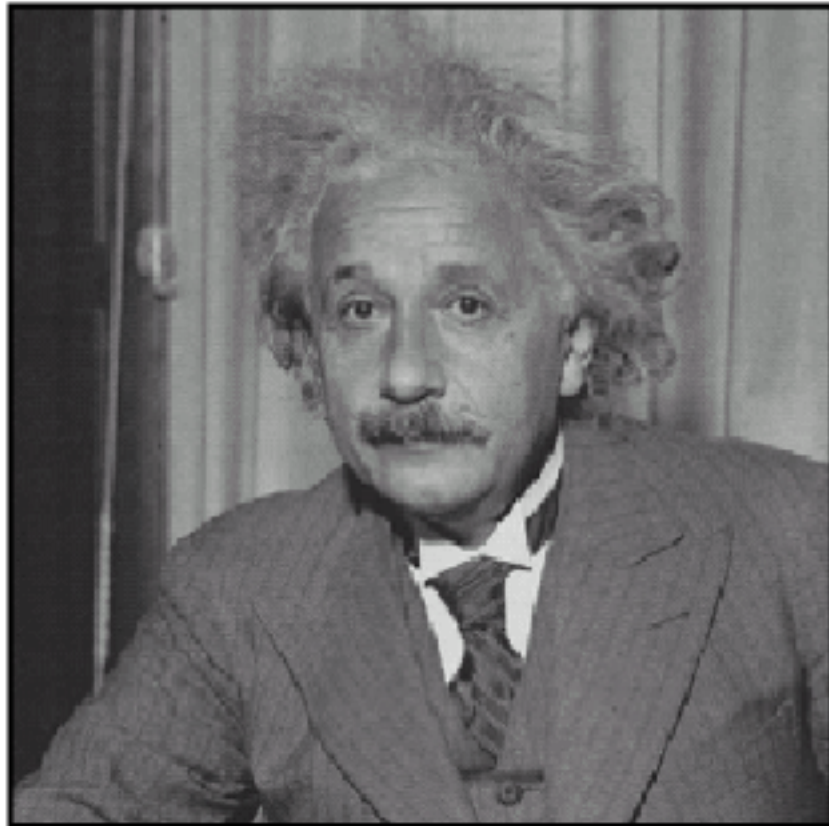
$\frac{1}{9}$

1	1	1
1	1	1
1	1	1

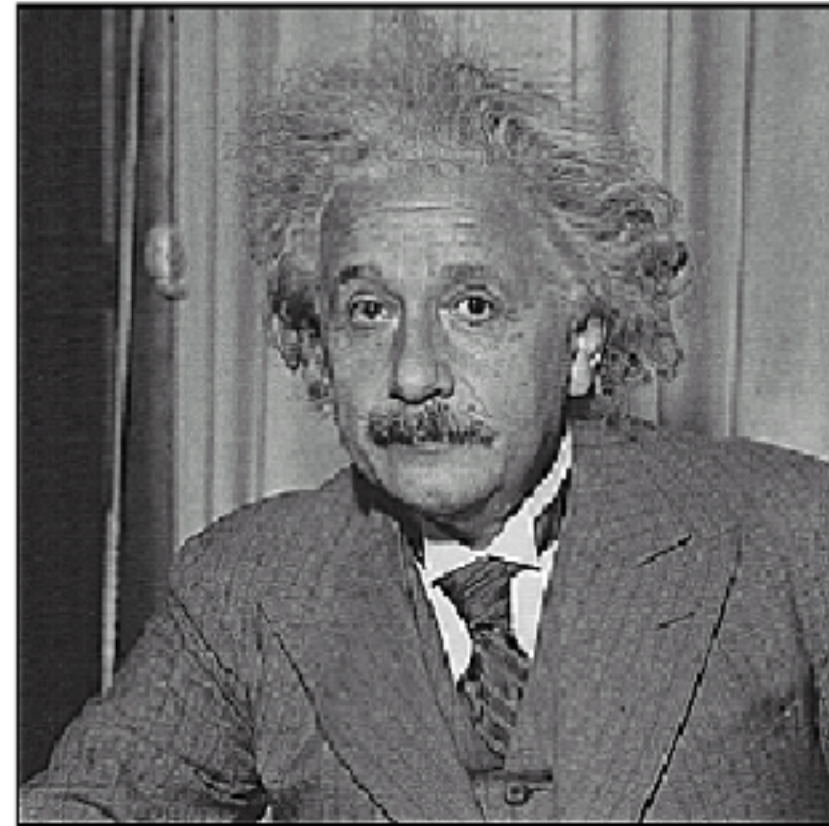
?

(Note that filter sums to 1)

Sharpening

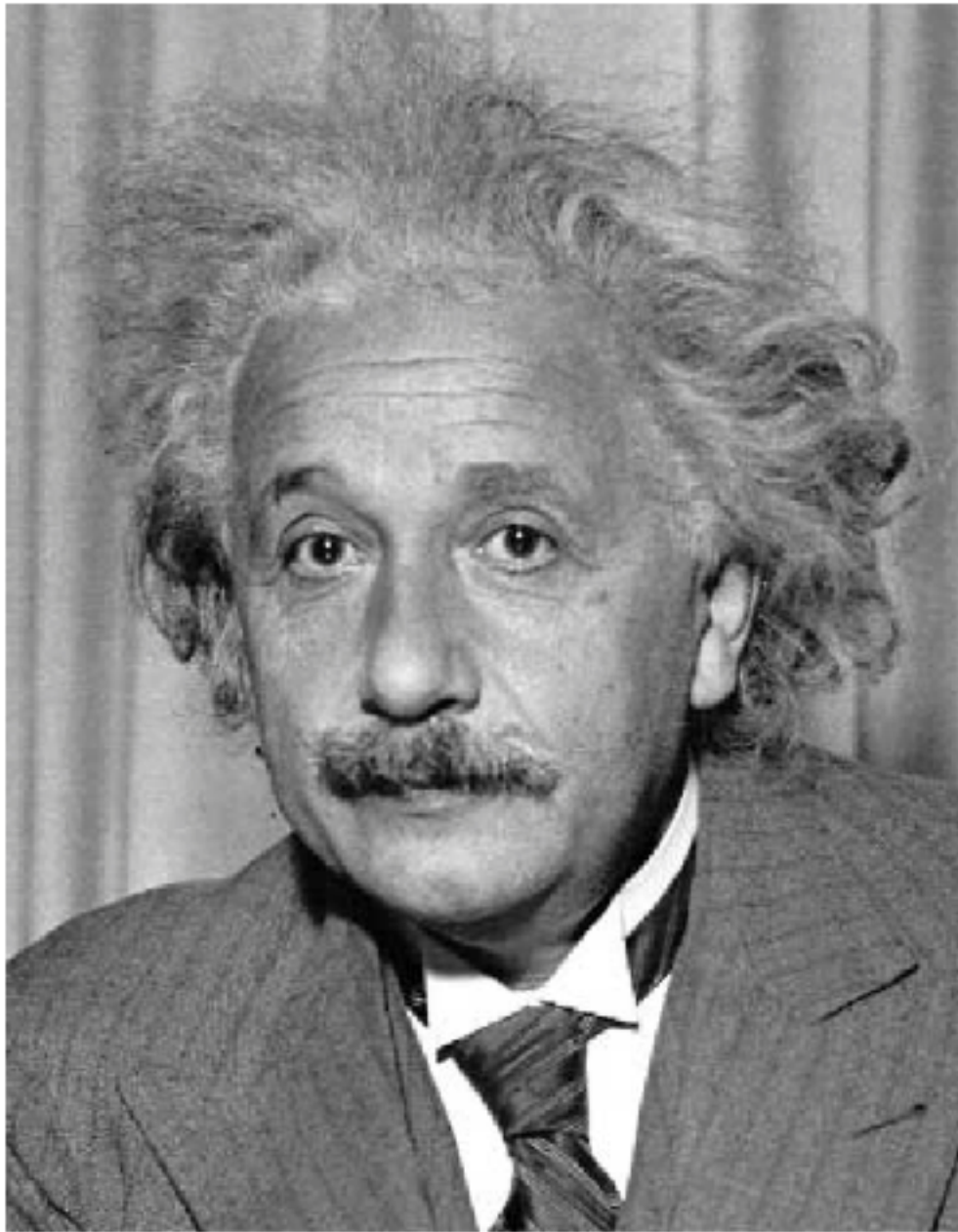


before



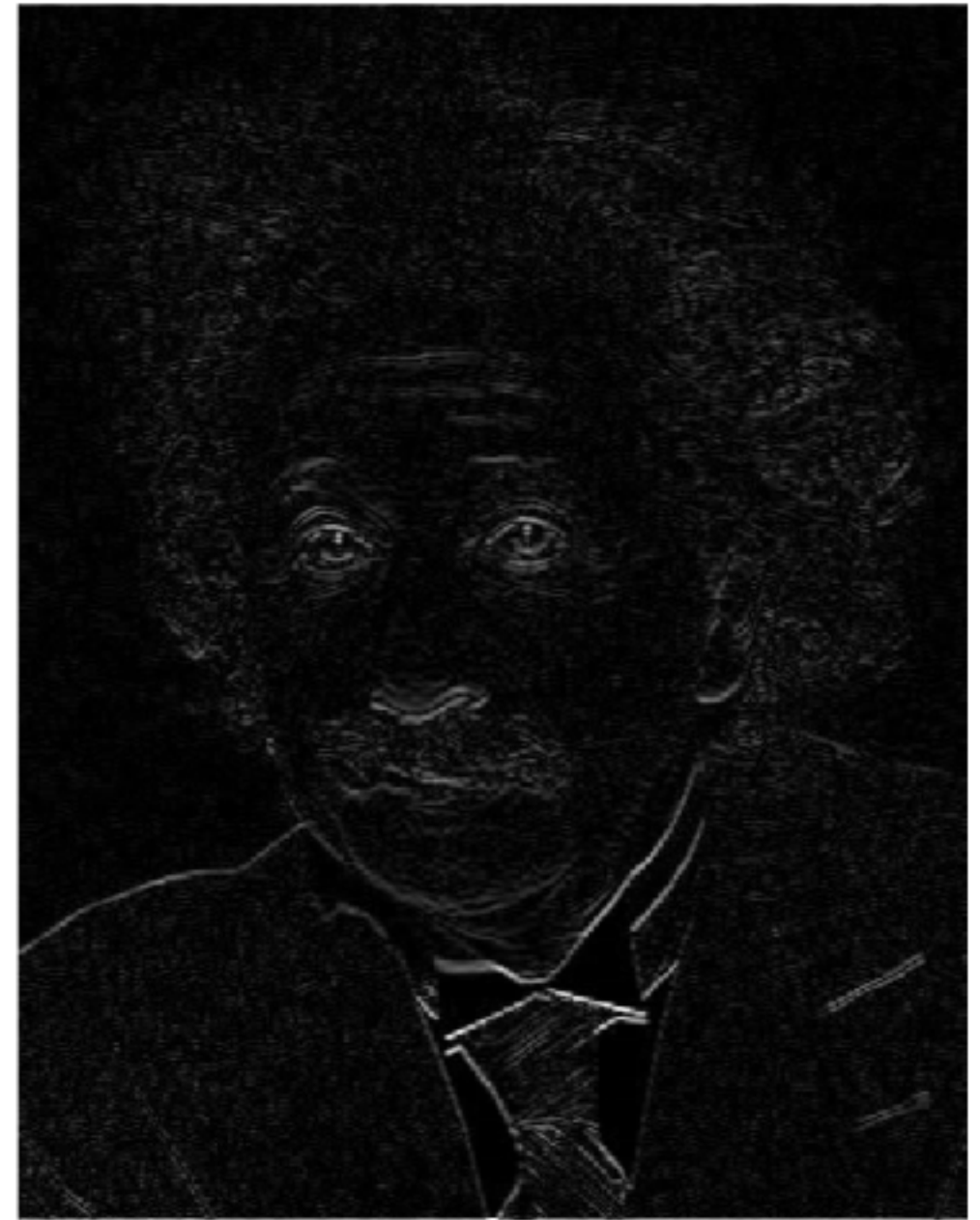
after

Other filters



1	2	1
0	0	0
-1	-2	-1

Sobel



Horizontal Edge
(absolute value)

Key properties of linear filters

Linearity:

$$\text{filter}(f_1 + f_2) = \text{filter}(f_1) + \text{filter}(f_2)$$

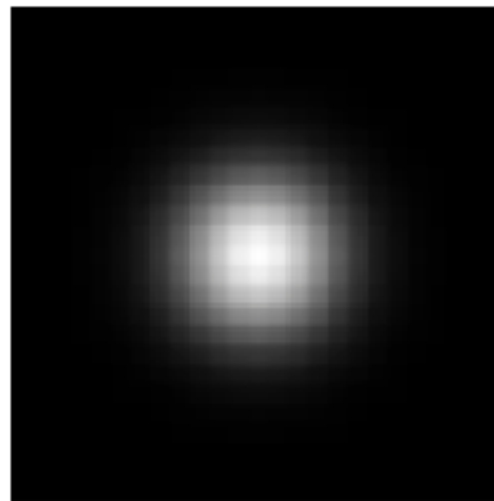
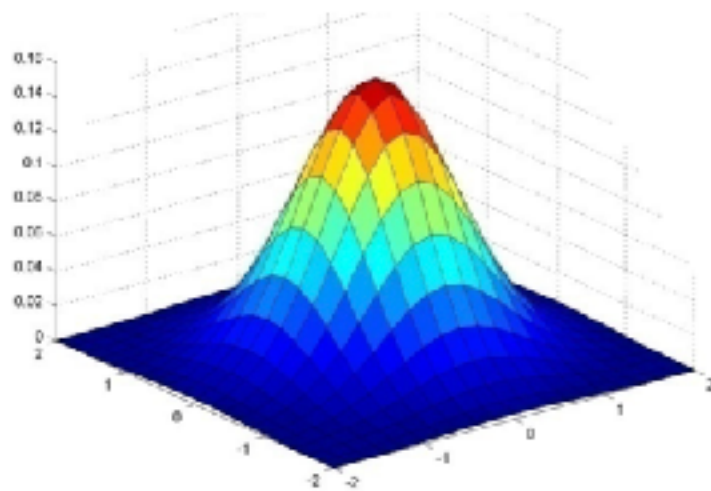
Shift invariance: same behavior regardless of pixel location

$$\text{filter}(\text{shift}(f)) = \text{shift}(\text{filter}(f))$$

Any linear, shift-invariant operator can be represented as a convolution

Important filter: Gaussian

- Weight contributions of neighboring pixels by nearness

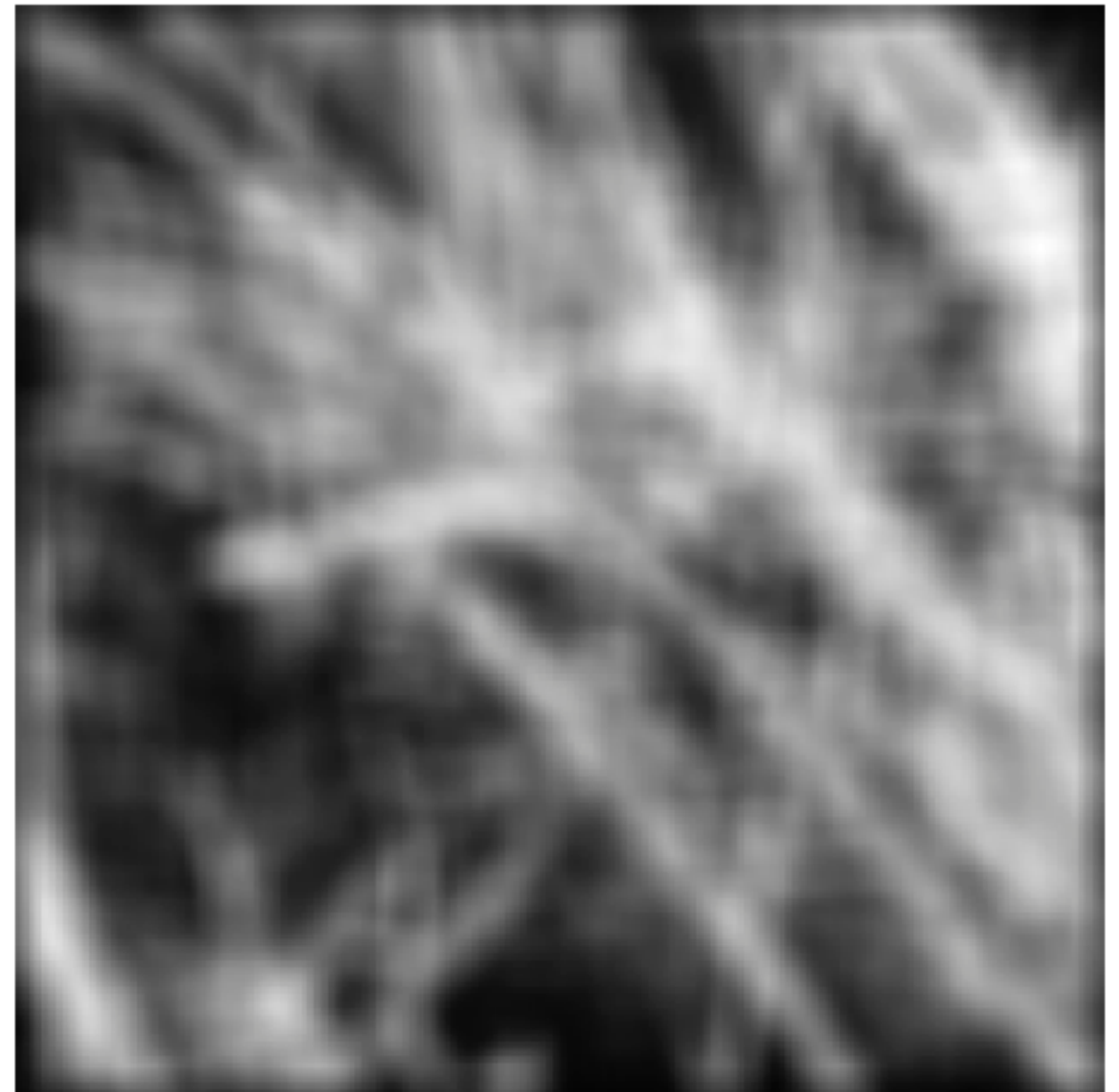


0.003	0.013	0.022	0.013	0.003
0.013	0.059	0.097	0.059	0.013
0.022	0.097	0.159	0.097	0.022
0.013	0.059	0.097	0.059	0.013
0.003	0.013	0.022	0.013	0.003

5 x 5, $\sigma = 1$

$$G_{\sigma} = \frac{1}{2\pi\sigma^2} e^{-\frac{(x^2+y^2)}{2\sigma^2}}$$

Smoothing with box filter



Practical matters

How big should the filter be?

- Values at edges should be near zero
- Rule of thumb for Gaussian: set filter half-width to about 3σ

- ▶ Remarks on linear filters vs. non-linear filters
- ▶ Feature detection
- ▶ Keypoint descriptors
- ▶ Keypoint matching